.Net CRUD Operation using Entity Framework using

Entity Framework (EF) is an **Object-Relational Mapper (ORM)** for .NET that allows developers to interact with a database using C# objects. Instead of writing raw SQL queries, Entity Framework enables you to use LINQ (Language Integrated Query) to query and manipulate data, making the process of interacting with a database much more intuitive and less error-prone.

Here's a breakdown of the main concepts and features of Entity Framework:

1. ORM (Object-Relational Mapper)

- EF provides a way to map C# objects (entities) to database tables. It eliminates the need for manually writing SQL to interact with the database.
- It allows developers to use object-oriented programming (OOP) concepts to perform CRUD (Create, Read, Update, Delete) operations on the data.

2. Database Models

- Entity Framework works with **models** (classes in C#) that represent tables in the database.
- These models are defined using C# classes, and EF automatically maps them to the underlying database structure.

3. DbContext

- The DbContext class is the primary class that interacts with the database in EF. It is used to query and save data to the database.
- It includes **DbSet<TEntity>** properties that represent collections of entities in the database (tables).

Advantages of Entity Framework:

- **Productivity**: It simplifies database access by abstracting away most of the SQL logic.
- **Maintainability**: Models and relationships are easier to maintain and update through code.
- **Portability**: EF Core allows you to build cross-platform applications.
- **Security**: By using parameterized queries, EF helps prevent SQL injection attacks.

Entity Framework makes database interaction in C# applications much more efficient and developer-friendly. It provides a rich set of features that simplify tasks like querying, updating, and managing data while allowing you to work in an object-oriented paradigm.

RESTful

RESTful" refers to a design style used to build web services that follow the principles of **REST** (Representational State Transfer). REST is an architectural style for designing networked applications, and when an API (Application Programming Interface) is described as "RESTful," it means the API adheres to these principles, making it simple, scalable, and easy to interact with.

Key Concepts of REST:

1. **Stateless**:
    - Every HTTP request from a client to a server must contain all the information the server needs to understand and process the request. The server doesn't store any information about previous requests.
    - Each request is independent and must be self-contained.
2. **Client-Server Architecture**:
    - The client and server are separate entities. The client makes requests to the server, and the server provides responses, but the two do not rely on each other's internal workings. This separation allows for more flexibility in the system.
3. **Uniform Interface**:
    - REST defines a set of conventions for communication between the client and server, typically using **HTTP** methods like GET, POST, PUT, DELETE, etc., to interact with resources.
    - These conventions ensure a standardized way of accessing and manipulating data.
4. **Resources**:
    - In REST, **resources** are the central objects that can be accessed or manipulated. A resource could be anything like a user, an order, a product, etc.
    - Each resource is identified by a **URI (Uniform Resource Identifier)**. For example, https://api.example.com/users could represent a collection of users, and https://api.example.com/users/1 could represent a single user with the ID 1.
5. **Representations**:
    - Resources are typically represented in formats like **JSON** or **XML**, and the client interacts with these representations. For example, a GET request to a user's resource might return the user's information in JSON format.
6. **Stateless Communication**:
    - RESTful APIs are stateless, meaning each request from a client contains all the data needed to understand the request (including authentication tokens, data payloads, etc.).

7. **Cacheable**:
    - Responses from the server can be marked as **cacheable** or **non-cacheable**. This allows clients to cache responses and reduce server load.

HTTP Methods in RESTful APIs:

- **GET**: Retrieves data from the server (e.g., retrieve a list of users).
- **POST**: Creates new data on the server (e.g., create a new user).
- **PUT**: Updates existing data on the server (e.g., update a user's details).
- **DELETE**: Deletes data from the server (e.g., remove a user).
- **PATCH**: Partially updates data on the server (e.g., update only the user's email).

Example of RESTful API Design:

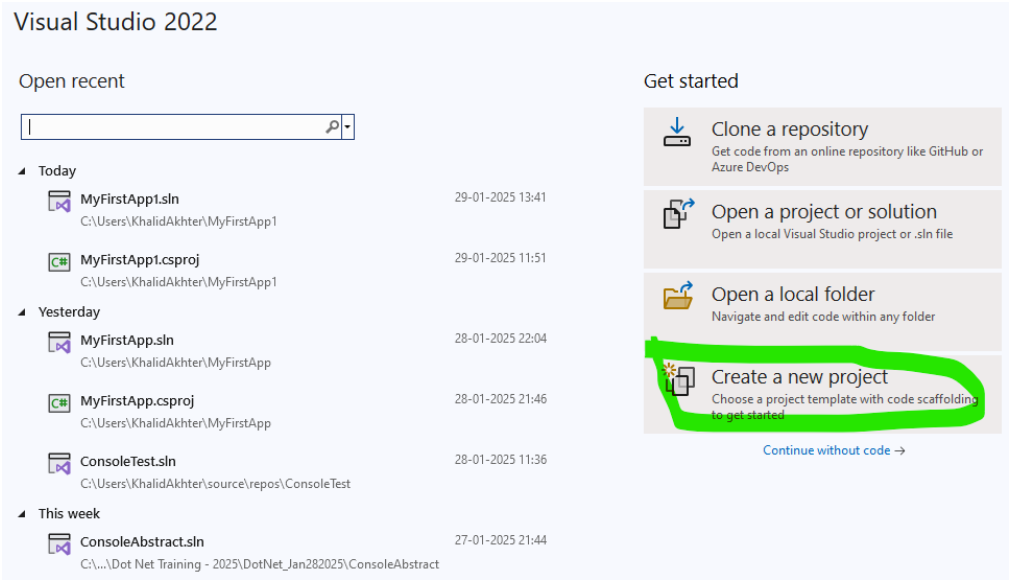
Assume you're working with a system that manages "users" and you have a RESTful API. The URIs and actions might look like this:

HTTP Method	URI	Action	Description
GET	/users	Retrieve all users	Fetches a list of all users.
GET	/users/{id}	Retrieve a specific user	Fetches details for the user with ID {id}.
POST	/users	Create a new user	Adds a new user to the system.
PUT	/users/{id}	Update an existing user	Modifies the user with ID {id}.
DELETE	/users/{id}	Delete a user	Removes the user with ID {id} from the system.

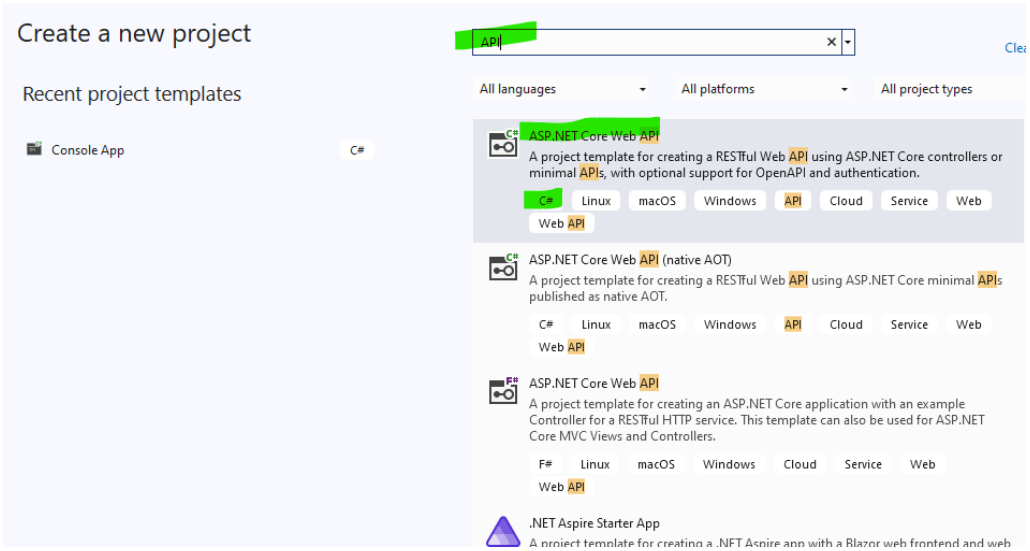
CRUD OPERATION

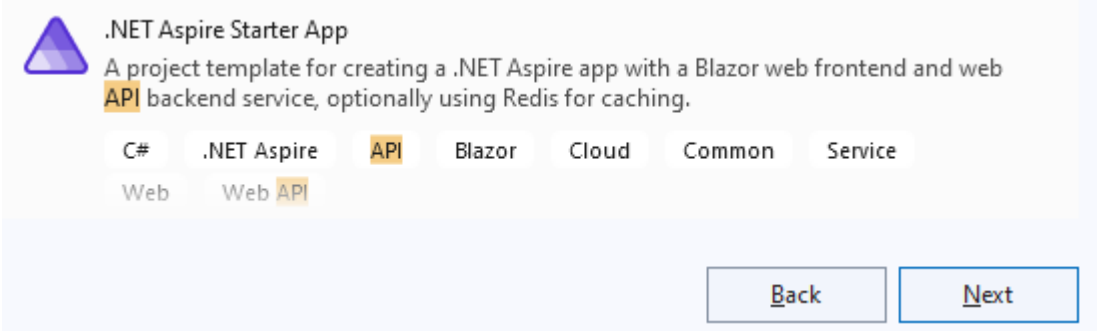
Crud Operation:

Step 1: Open Visual Studio 2022



Select: Create a new project.



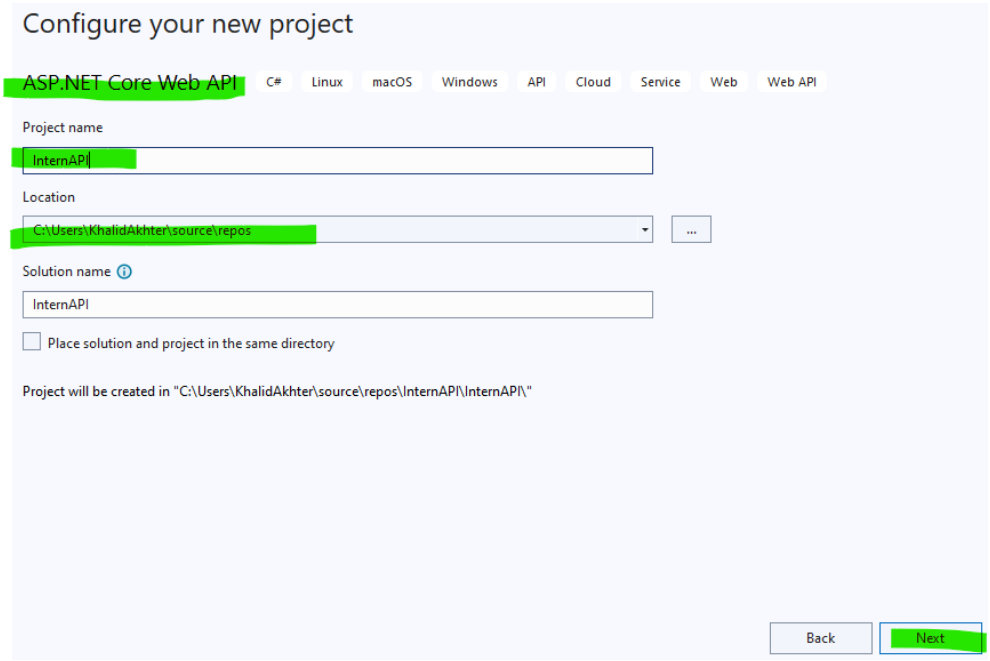


Search API

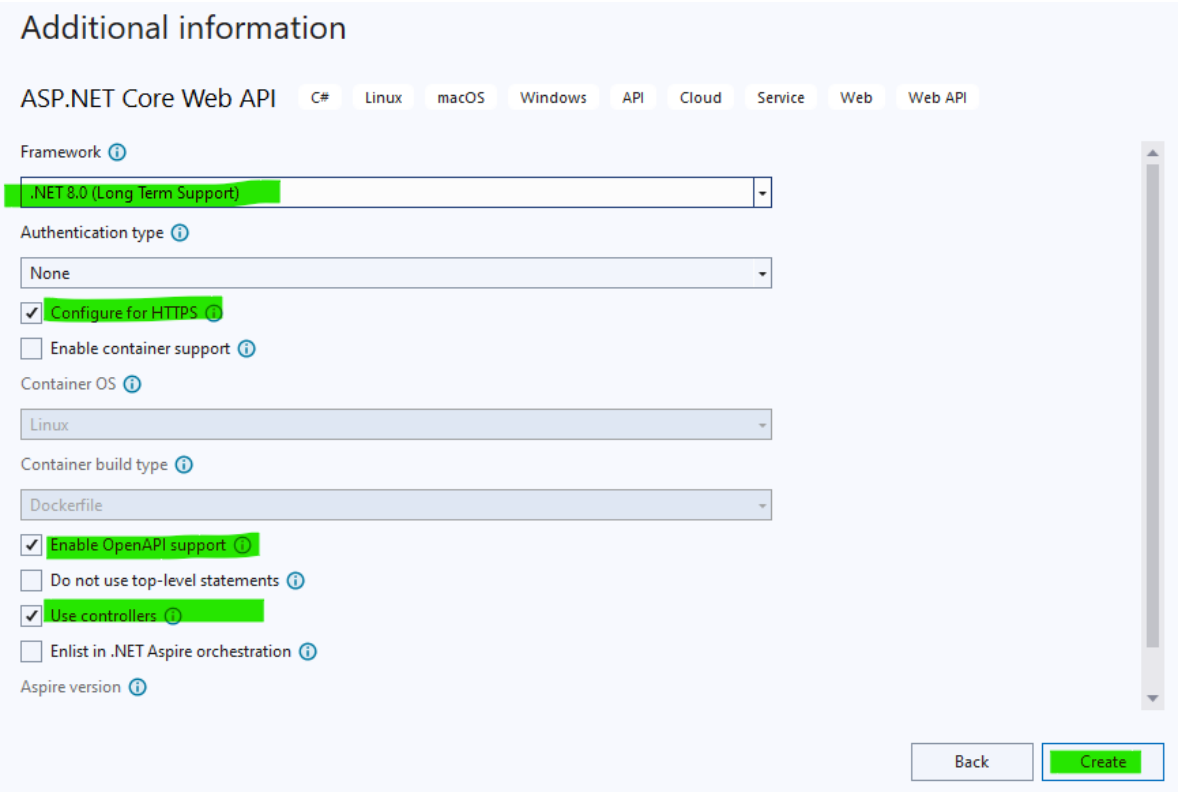
Select: ASP.Net Core Web API

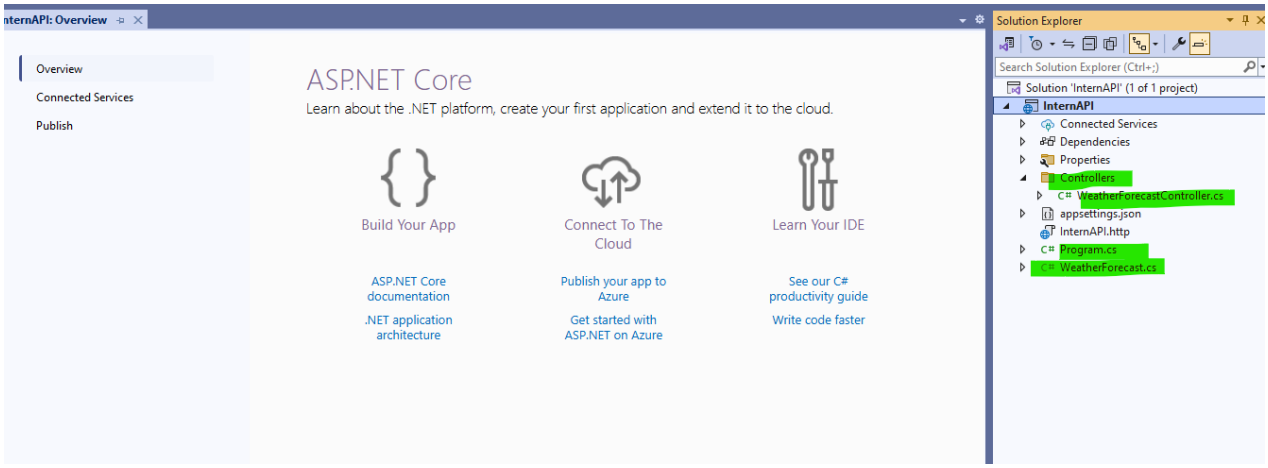
Language should be C#.

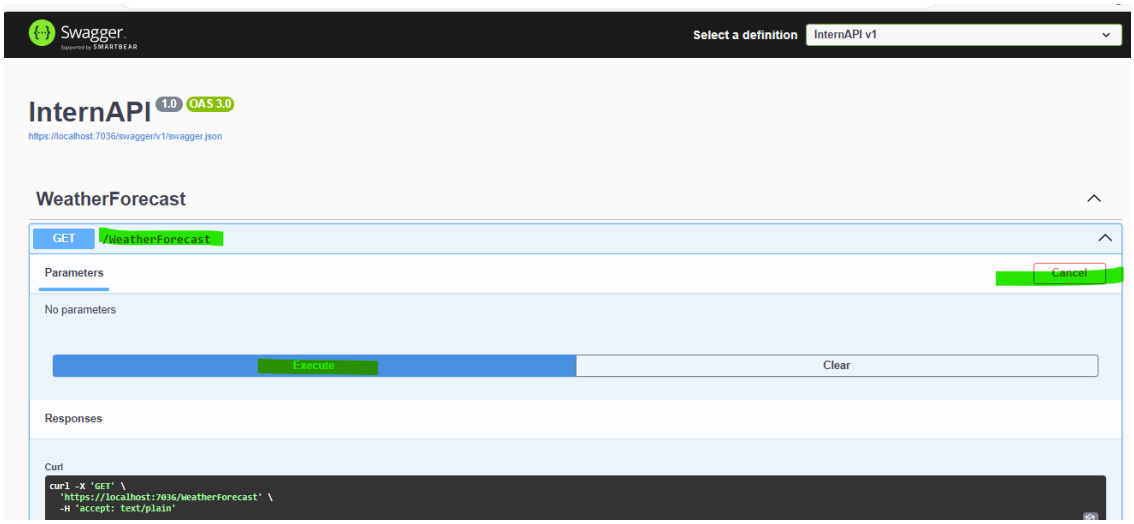
Click on Next



Click on Next:







Create Table in SQL Server:

Create Table Students(id int not null primary key identity(1,1),

StudentName nvarchar(100),

StudentGender nvarchar(20),

Age int,

Standard int,

FatherName nvarchar(100)

)

Insert some records:

insert into Students(StudentName,StudentGender,Age,Standard,FatherName)

Values

('Rohit Raj','Male',20,12,'Unknown'),

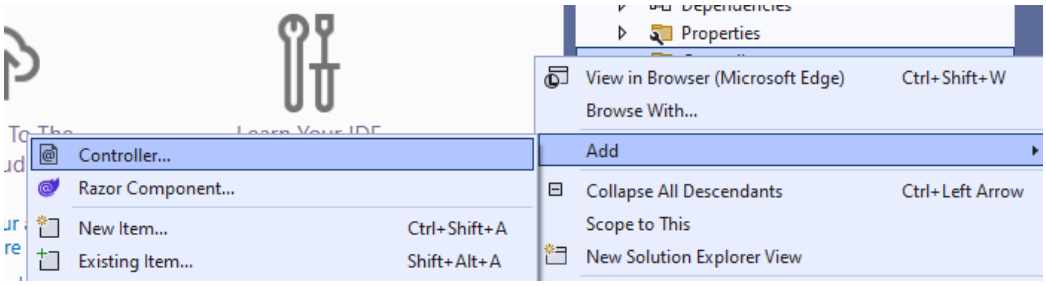
('Anil','Male',22,10,'Rajesh'),

('Amrita','Female',18,8,'Rajendra')

Go to VS 2022:

Create one new Controller as StudentController:

Right click on Controller folder



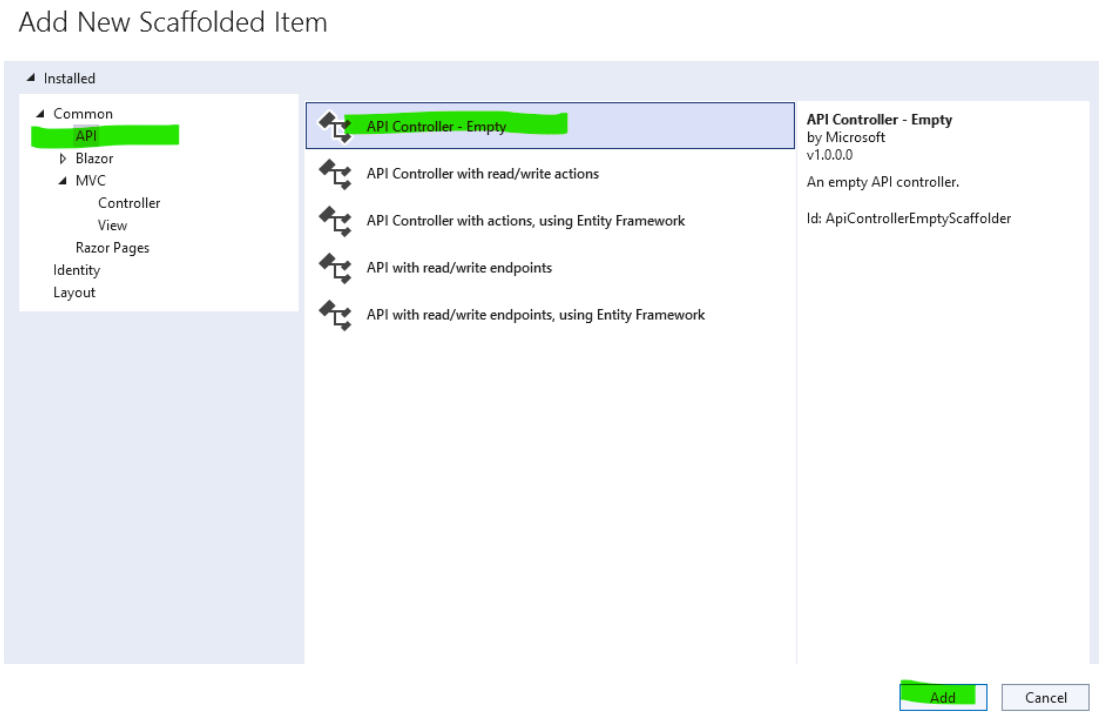
Click on Add

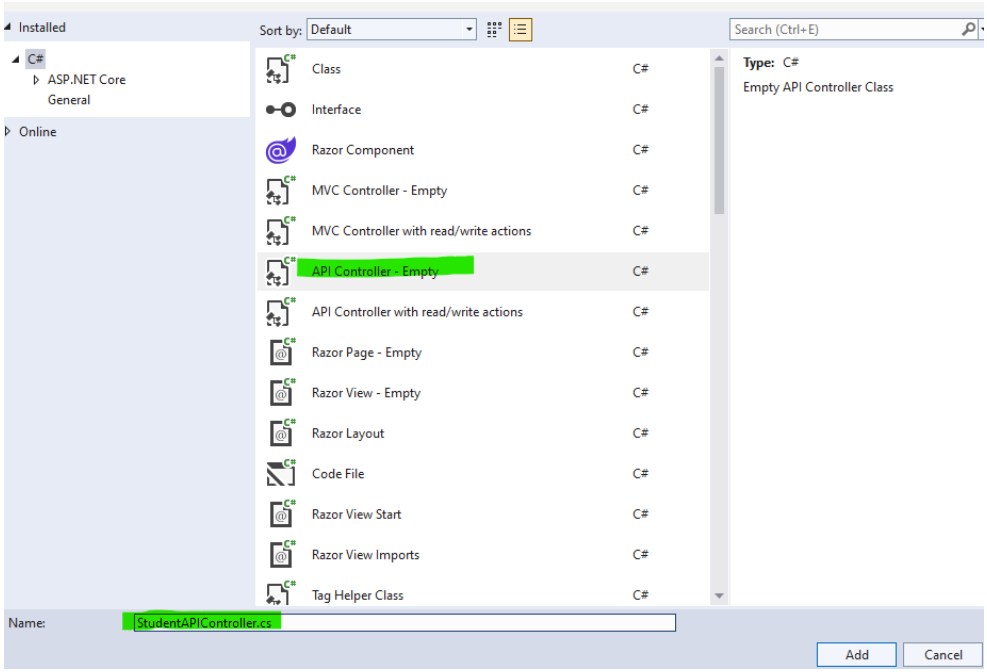
Click on Controller

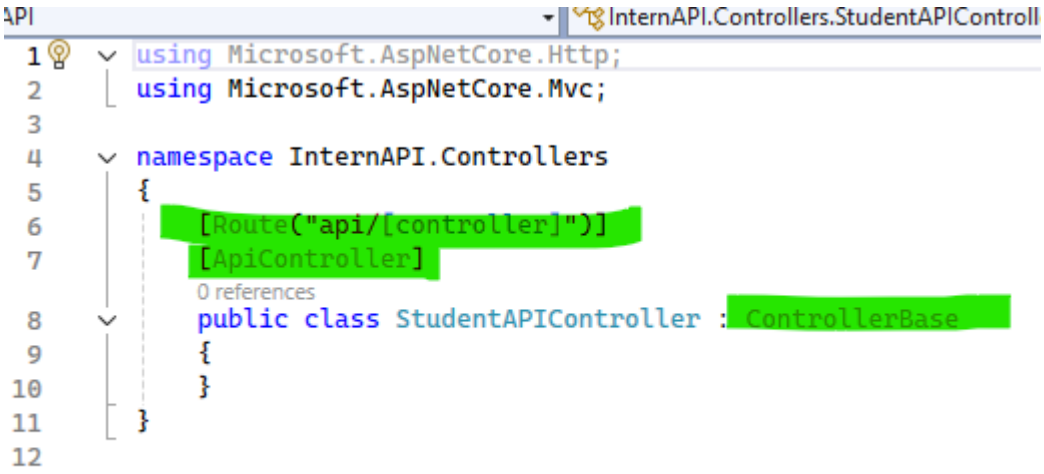
Click on API

Select API Controller – Empty

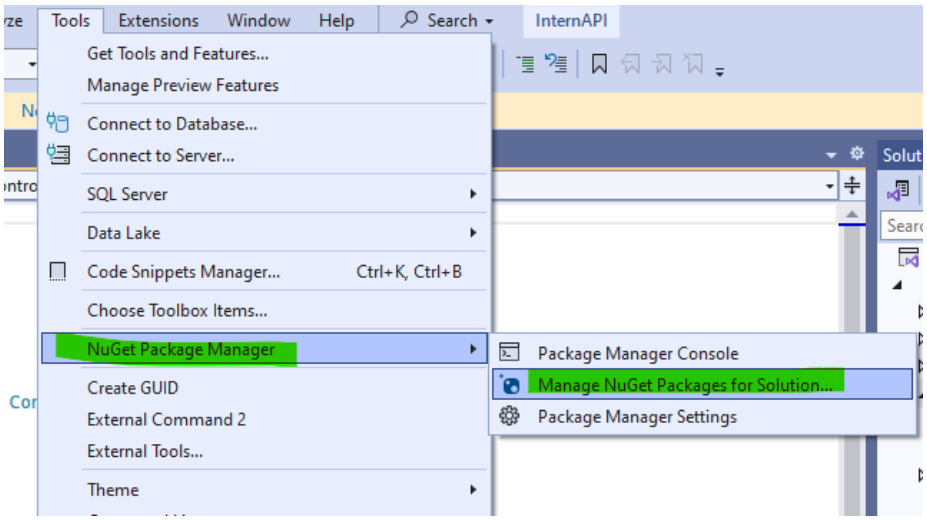
Click on Add.







Open Nuget window:

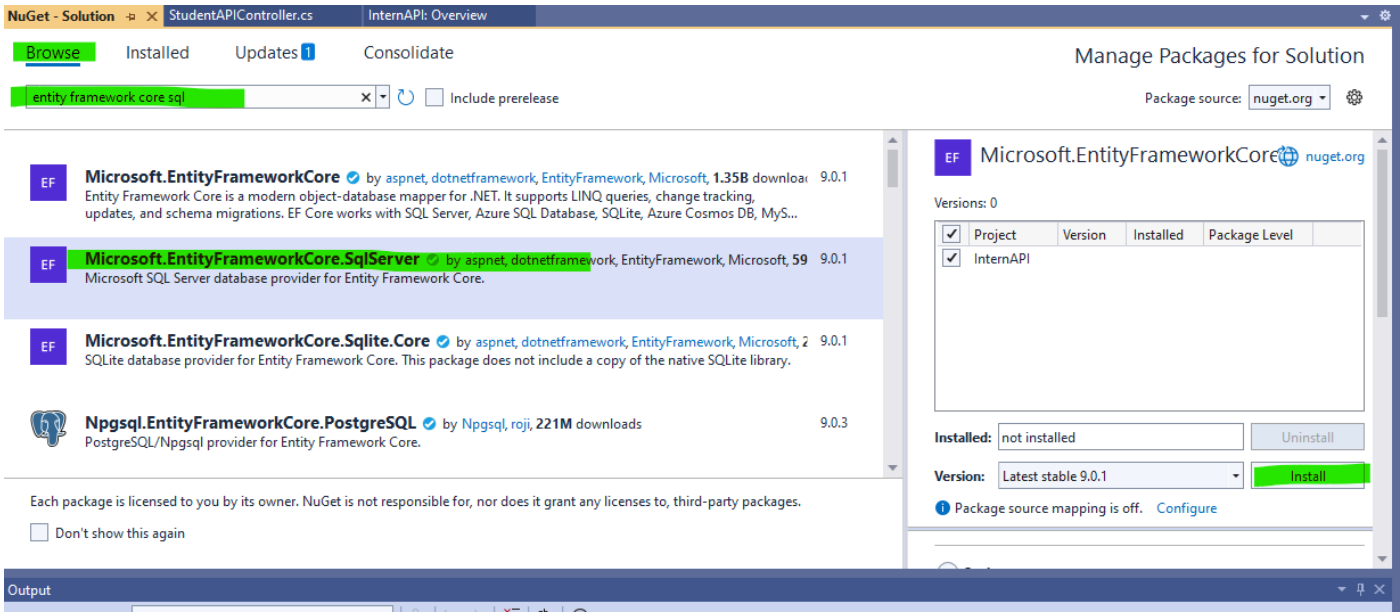


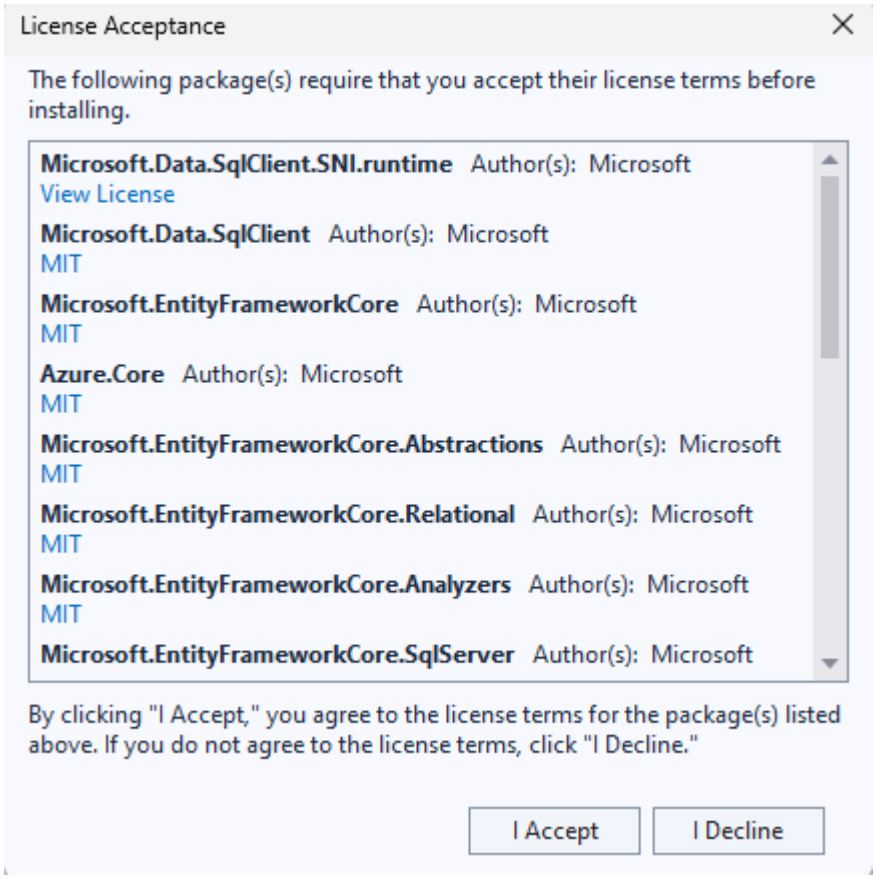
Browse

Search for: entity framework core sql

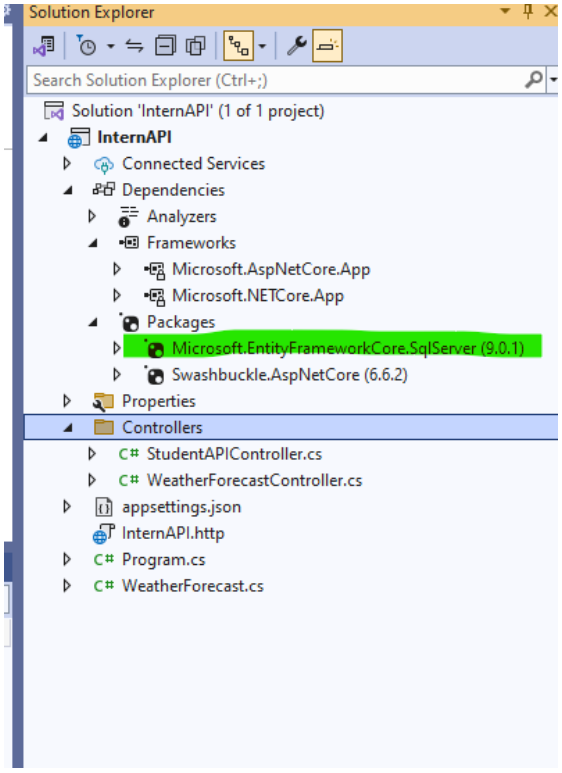
Select Microsoft.EntityFrameCore.SqlServer

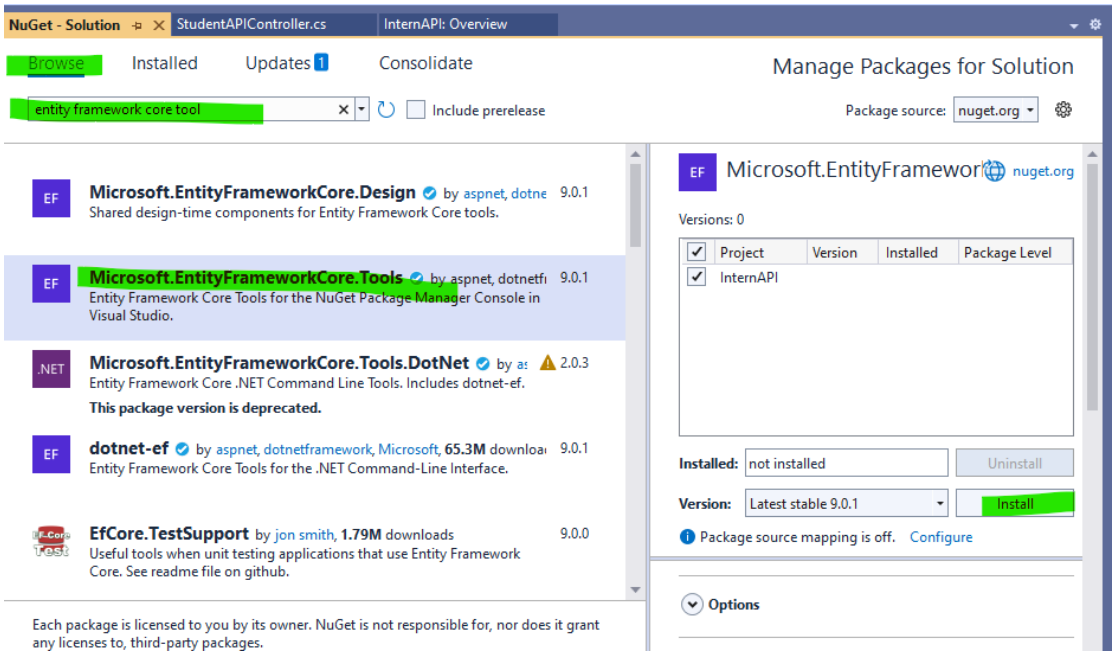
Install





Click on I Accept



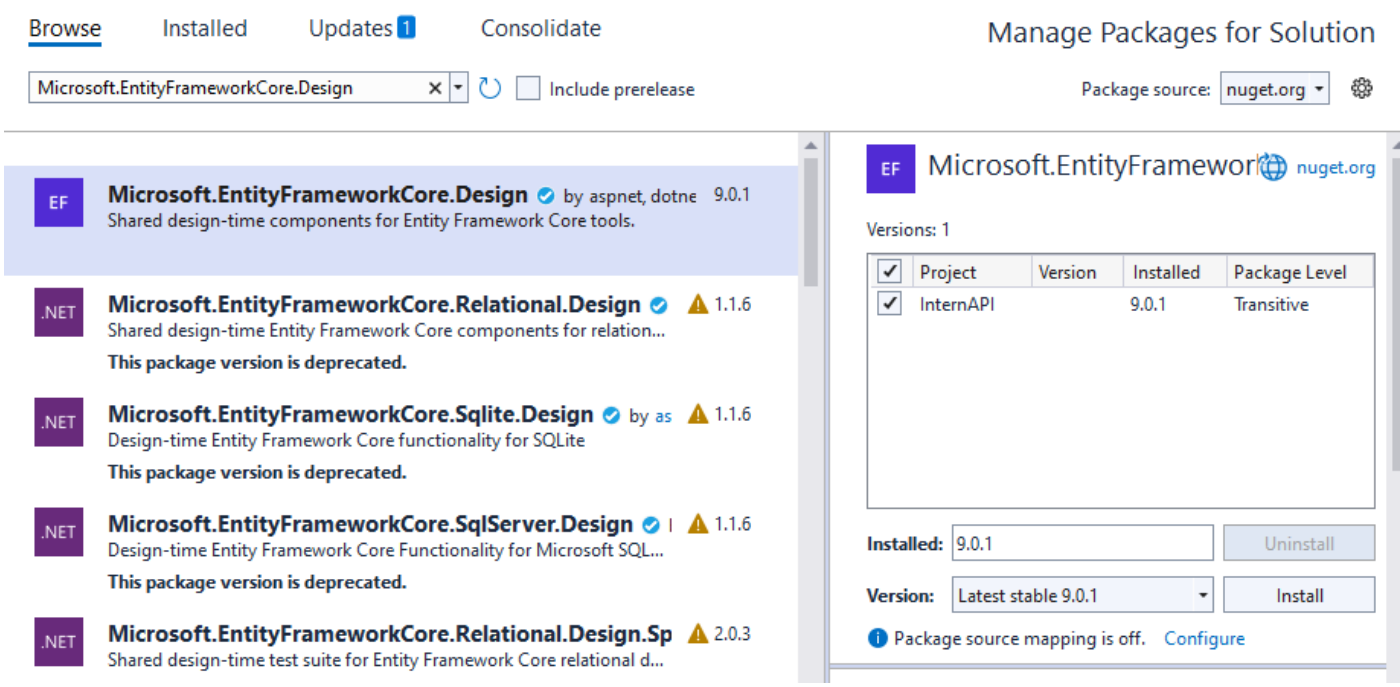


Click on Install – Apply – I Accept

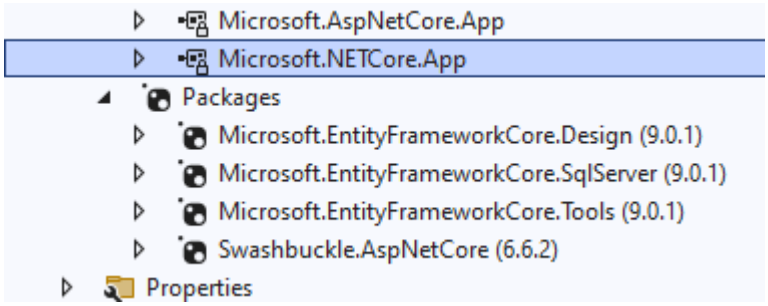
It will install Microsoft.EntityFrameworkCore.Tools

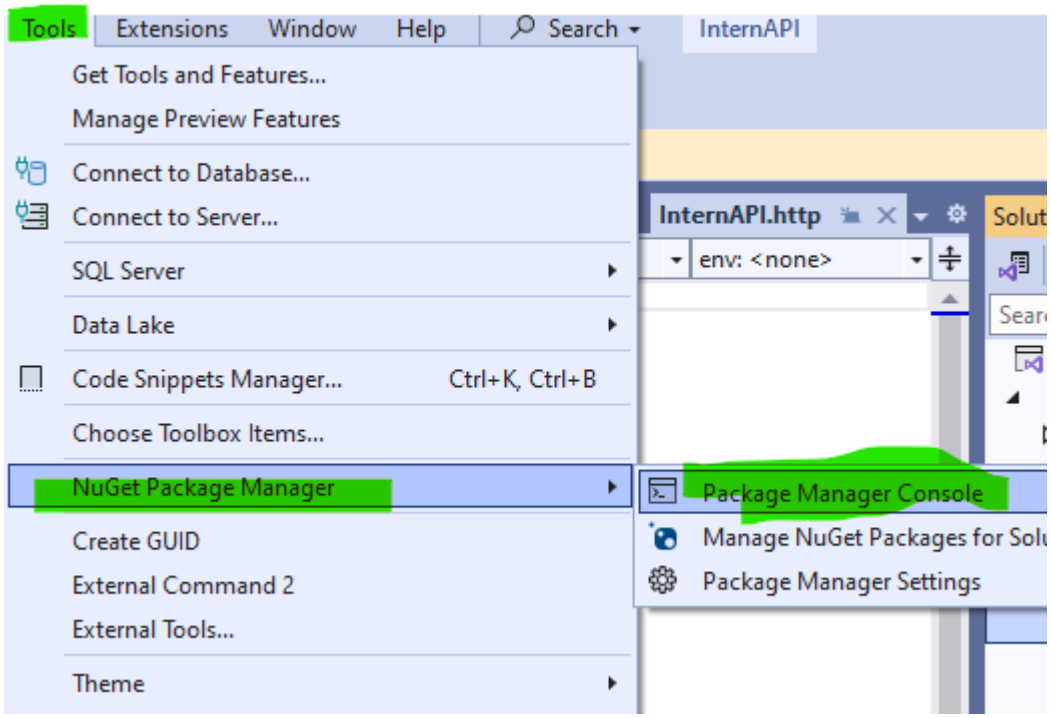
Similarly install:

Microsoft.EntityFrameworkCore.Design


Browse
Installed
Updates 1
Consolidate
Manage Packages for Solution
Microsoft.EntityFrameworkCore.Design
Include prerelease
Package source: nuget.org
Microsoft.EntityFrameworkCore.Design by aspnet, dotne 9.0.1
Shared design-time components for Entity Framework Core tools.
Microsoft.EntityFrameworkCore.Relational.Design 1.1.6
Shared design-time Entity Framework Core components for relation...
This package version is deprecated.
Microsoft.EntityFrameworkCore.Sqlite.Design by as 1.1.6
Design-time Entity Framework Core functionality for SQLite
This package version is deprecated.
Microsoft.EntityFrameworkCore.SqlServer.Design 1.1.6
Design-time Entity Framework Core Functionality for Microsoft SQL...
This package version is deprecated.
Microsoft.EntityFrameworkCore.Relational.Design.Sp 2.0.3
Shared design-time test suite for Entity Framework Core relational d...
Microsoft.EntityFramewor nuget.org
Versions: 1
Project
Version
Installed
Package Level
InternAPI
9.0.1
Transitive
Installed: 9.0.1
Uninstall
Version: Latest stable 9.0.1
Install
Package source mapping is off. Configure

Finally:


Microsoft.AspNetCore.App
Microsoft.NETCore.App
Packages
Microsoft.EntityFrameworkCore.Design (9.0.1)
Microsoft.EntityFrameworkCore.SqlServer (9.0.1)
Microsoft.EntityFrameworkCore.Tools (9.0.1)
Swashbuckle.AspNetCore (6.6.2)
Properties


Tools
Extensions
Window
Help
Search
InternAPI
Get Tools and Features...
Manage Preview Features
Connect to Database...
Connect to Server...
SQL Server
Data Lake
Code Snippets Manager... Ctrl+K, Ctrl+B
Choose Toolbox Items...
NuGet Package Manager
Create GUID
External Command 2
External Tools...
Theme
InternAPI.http
env: <none>
Package Manager Console
Manage NuGet Packages for Solu
Package Manager Settings

Going to install EF DbContext

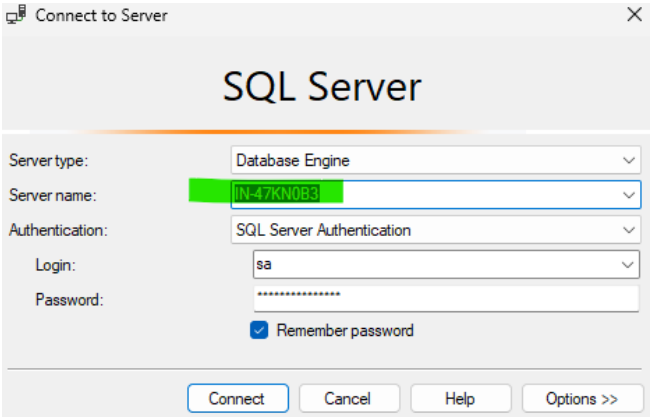
Note:

The Scaffold-DbContext command is used in **Entity Framework Core** to generate C# model classes and a DbContext based on an existing database. This is part of the **Database-First** approach, where you work with an existing database and generate the models and context automatically rather than writing them by hand.

Syntax of Scaffold-DbContext:

Scaffold-DbContext "Server= IN-47KN0B3;Database=Test;Trusted_Connection=True;" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models

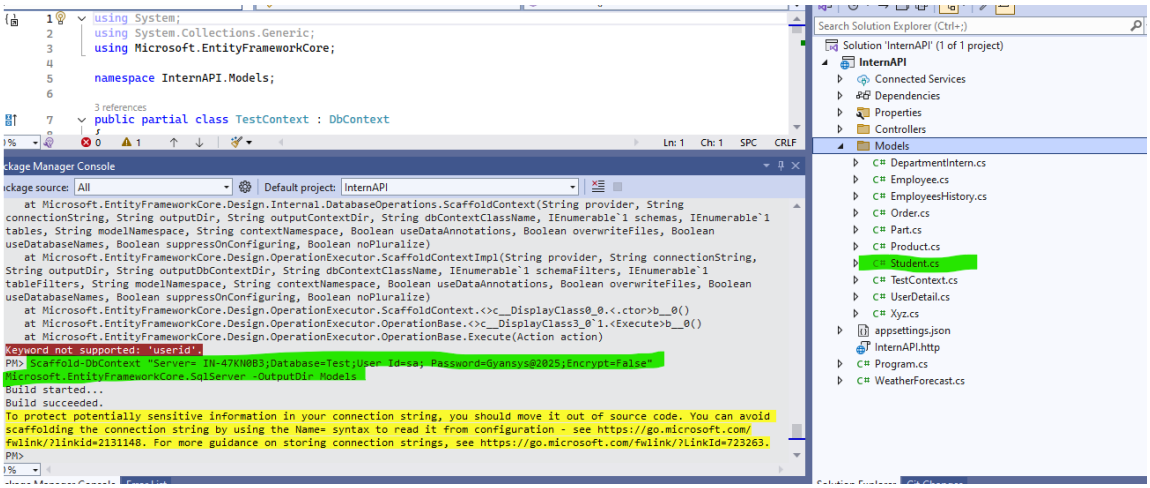
How to get server name:

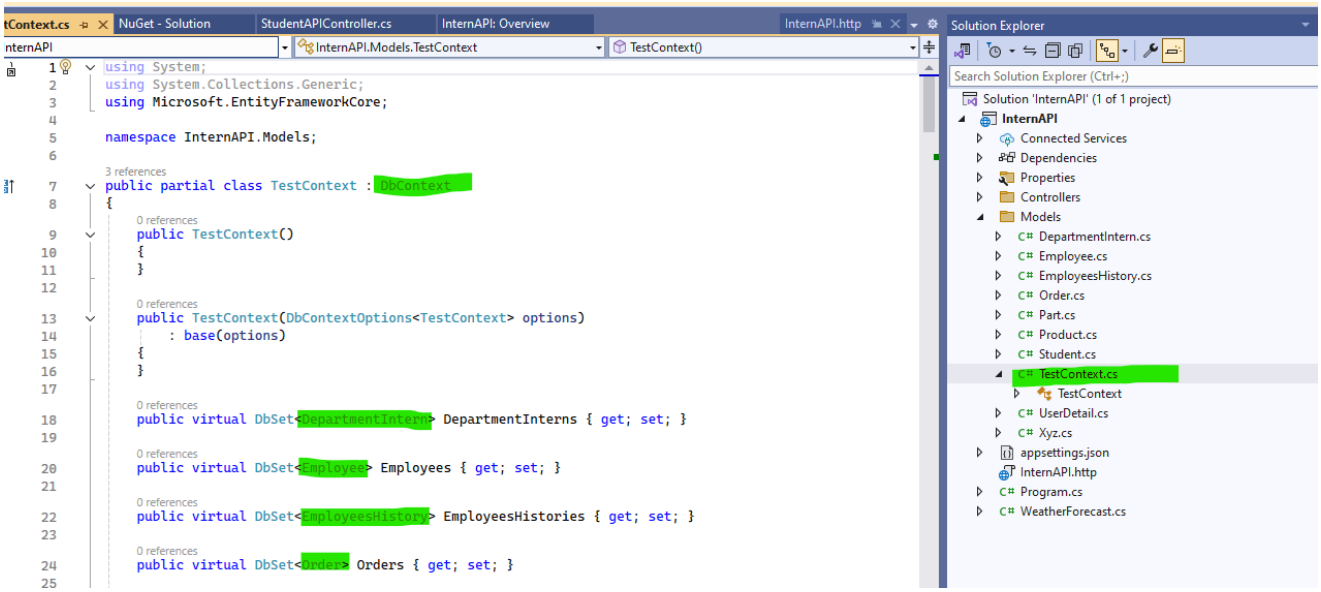


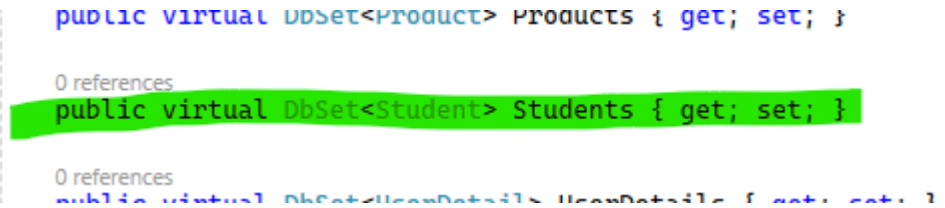
Create one folder as Models:

Run the below command in Nuget Package Manger Console

Scaffold-DbContext "Server= IN-47KN0B3;Database=Test;User Id=sa; Password=Gyansys@2025;Encrypt=False" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models



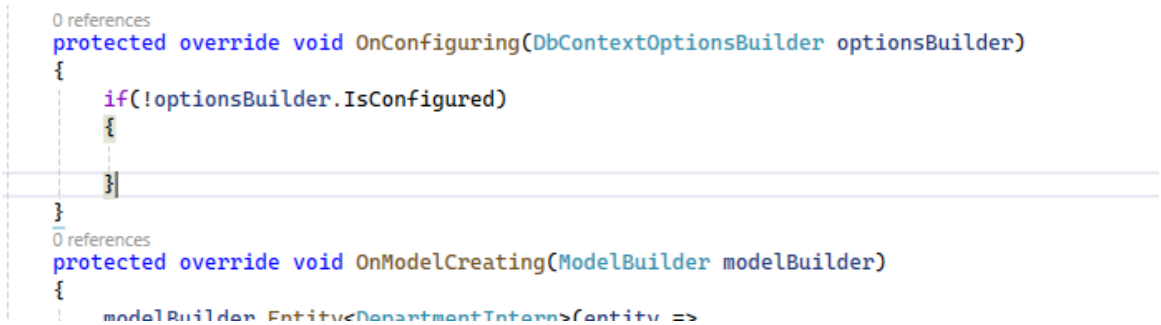




Go to appsetting.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "dbcs": "Server= IN-47KN0B3;Database=Test;User Id=sa;
Password=Gyansys@2025;Encrypt=False;"
  },
  "AllowedHosts": "*"
}
```

Remove from TestContext.cs



Register the TestContext in program.cs file, this is sql server provider

var provider = builder.Services.BuildServiceProvider();

var config = provider.GetRequiredService<IConfiguration>();

builder.Services.AddDbContext<TestContext>(item => item.UseSqlServer(config.GetConnectionString("dbcs")));

Above: var app = builder.Build();

Go to StudentAPIController:

Create constructor and implement DI:

```
namespace InternAPI.Controllers
{
  [Route("api/[controller]")]
  [ApiController]
  public class StudentAPIController : ControllerBase
  {
    private readonly TestContext context;
    public StudentAPIController(TestContext context)
    {
      this.context = context;
```

```
    }
  }
}
```

Creating Endpoint GetStudents:

```
[HttpGet]
 public async Task<ActionResult<List<Student>>> GetStudents()
{
   var data = await context.Students.ToListAsync();
   return Ok(data);
 }
```

Create one more Endpoint GetStudentById

```
[HttpGet("{id}")]
 public async Task<ActionResult<List<Student>>> GetStudentById(int id)
{
   var student = await context.Students.FindAsync(id);
   if(student==null)
   {
      return NotFound();
   }
   return Ok(student);
 }
```

Create one moe Endpoint CreateStudent

```
[HttpPost]
 public async Task<ActionResult<List<Student>>> CreateStudent(Student std)
{
   await context.Students.AddAsync(std);
   await context.SaveChangesAsync();
   return Ok(std);
 }
```

Create one moe Endpoint UpdateStudent

```
[HttpPut("{id}")]
public async Task<ActionResult<List<Student>>> UpdateStudent(int id, Student std)
{
   if(id!=std.Id)
   {
      return BadRequest();
   }
   context.Entry(std).State = EntityState.Modified;
   await context.SaveChangesAsync();
   return Ok(std);
}
```

Create one moe Endpoint DeleteStudent

```
[HttpDelete("{id}")]
public async Task<ActionResult<List<Student>>> DeleteStudent(int id)
{
   var std = await context.Students.FindAsync(id);
   if(std==null)
   {
      return NotFound();
   }
   context.Students.Remove(std);
   await context.SaveChangesAsync();
   return Ok(std);
}
```

==========Happy CRUD Operation==========